SANJIVANI RURAL EDUCATION SOCIETY'S

SANJIVANI COLLEGE OF ENGINEERING, KOPARGAON – 423603

(An Autonomous Institute Affiliated to Savitribai Phule Pune University, Pune)

NAAC 'A' Grade Accredited

NBA Accredited – UG Programme

DEPARTMENT OF INFORMATION TECHNOLOGY

MINI PROJECT REPORT

on

"ScholarMitra AI – Intelligent Scholarship Discovery, Eligibility Prediction and Personalized Financial Aid Recommendation System"

Submitted by:

Roll No	PRN	Name	Signature
08	0124UITF1008	Malkar Tithi Vinod	
13	0124UITM1013	Kshirsagar Yashraj Chandrakant	
16	0124UITF1016	Shinde Bhakti Santosh	
161	0125UITF2013	Kulkarni Neha Yogesh	

Class: T.Y. B. Tech Information Technology

Division: A

Subject: Machine Learning Laboratory

Academic Year: 2026-2027

Prof. K. D. Patil
Course In-Charge

Dr. M. A. Jawale
Head of Department

Index

Sr. No	Title	Page no.
1	Problem Statement	3
2	Dataset Description	3
3	Algorithm	5
4	Implementation	8
5	Results: Compare the Results of Two Algorithms	14
6	Performance Evaluation	15
7	Conclusion	17
8	References	17

2. Problem Statement

Scholarship schemes grant financial support to students on the basis of a number of conditions. In the dataset used for this project, a scholarship-related outcome may depend on several attributes, namely the Education Qualification, Gender, Community, Religion, Exservice-men status, Disability status, Sports status, Annual-Percentage, Income, India-related category and the Scholarship Name. Each record in the dataset represents one combination of these attributes together with the corresponding Outcome.

The dataset contains 245,760 records and 12 attributes. The attribute Outcome is the target variable, and the remaining 11 attributes are used as feature variables. Manually analysing scholarship-related conditions across such a large number of records is difficult, time-consuming and prone to error, because the Outcome is influenced by a combination of many attributes rather than by a single attribute.

To develop a machine learning classification model that can learn patterns from scholarship-related data and predict the Outcome for a given record.

3. Dataset Description

The project uses the dataset file dataset_combined.xlsx, and the data is read from the sheet named dataset_revised. The source of the dataset is Kaggle. The general characteristics of the dataset are given in Table 3.1.

Table 3.1: Dataset Summary

Property	Value
Dataset file	dataset_combined.xlsx
Sheet used	dataset_revised
Source	Kaggle
Number of records	245,760
Number of attributes	12
Number of feature variables	11
Target variable	Outcome

Missing values	0

Table 3.2: Attribute Description

No.	Attribute	Description
1	Name	Scholarship name
2	Education Qualification	Education qualification/category
3	Gender	Gender category
4	Community	Community category
5	Religion	Religion category
6	Exservice-men	Ex-servicemen status
7	Disability	Disability status
8	Sports	Sports status
9	Annual-Percentage	Annual percentage category
10	Income	Income category
11	India	India-related eligibility category
12	Outcome	Target variable

Outcome Distribution

The target variable Outcome has two classes, 0 and 1. The class percentages are calculated as (class count / 245,760) × 100: for Outcome 0, (212,992 / 245,760) × 100 = 86.67%, and for Outcome 1, (32,768 / 245,760) × 100 = 13.33%.

Table 3.3: Outcome Distribution

Outcome	Number of Records	Percentage
0	212,992	86.67%
1	32,768	13.33%
Total	245,760	100.00%

The dataset is class-imbalanced. Outcome 0 contains 212,992 records (86.67%), whereas Outcome 1 contains 32,768 records (13.33%). Outcome 0 therefore has 6.5 times as many records as Outcome 1. Because of this imbalance, a stratified train-test split is used and several evaluation metrics are considered instead of accuracy alone.

4. Algorithm

Two supervised classification algorithms are used in this project: the Decision Tree Classifier and the Random Forest Classifier. No other algorithm is used.

4.1 Decision Tree Classifier

The Decision Tree is a supervised classification algorithm that learns decision rules from the input features and uses them to predict the target class. The model is represented as a tree in which every internal node tests a feature, every branch represents the result of the test, and every leaf node gives a class label.

A Decision Tree is suitable for the scholarship classification dataset because all the attributes are categorical and scholarship conditions are naturally expressed as rules (for example, conditions on qualification, category, income or percentage). The algorithm handles encoded categorical data well, does not require feature scaling, and produces rules that are easy to interpret. The model is created using:

DecisionTreeClassifier(random_state=42)

4.2 Random Forest Classifier

The Random Forest is an ensemble classification algorithm that combines multiple decision trees and uses their combined predictions to decide the final class. Each tree is trained on a random sample of the training data and a random subset of features, and the final prediction is obtained by majority voting among the trees.

Random Forest is suitable for this dataset because it can model interactions between many categorical attributes, works with the large number of records (245,760) and the one-hot encoded features, and reduces the over-fitting tendency of a single tree by averaging many trees. The model is created using:

RandomForestClassifier(n_estimators=100,random_state=42,n_jobs=-1)

The parameter n_jobs=-1 is used only for efficient execution on the large dataset by using all available processor cores.

4.3 Experiment Settings

Table 4.1: Experiment Settings

Setting	Value
Dataset / Sheet	dataset_combined.xlsx / dataset_revised
Training size	80%
Testing size	20%
Random state	42
Stratification	Yes (on Outcome)
Categorical encoding	One-Hot Encoding
Feature scaling	Not used (not required for tree-based algorithms)
Algorithm 1	Decision Tree Classifier
Algorithm 2	Random Forest Classifier (100 estimators)

4.4 Methodology

The methodology followed in this project is shown below. A stratified 80% training and 20% testing split with random_state = 42 is used because the target classes are imbalanced. All 11 feature attributes are categorical, so One-Hot Encoding is applied. StandardScaler is not used, because Decision Tree and Random Forest are tree-based algorithms that do not require feature scaling.

Dataset

↓

Data Loading

↓

Data Inspection

↓

Feature and Target Separation

↓

Categorical Encoding (One-Hot Encoding)

↓

Training and Testing Split (80% / 20%, Stratified)

↓

Decision Tree Training

↓

Random Forest Training

↓

Predictions

↓

Confusion Matrix

↓

Accuracy

↓

Precision

↓

Recall

↓

F1-Score

↓

Algorithm Comparison

5. Implementation

The implementation is carried out in Python using the Pandas and Scikit-learn libraries. The steps of the implementation, the code used and the results obtained are given below.

5.1 Importing Required Libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (confusion_matrix, accuracy_score,
                             precision_score, recall_score, f1_score)
```

5.2 Loading the Dataset

```
df = pd.read_excel("dataset_combined.xlsx", sheet_name="dataset_revised")
print(df.shape)
```

Output: (245760, 12). The dataset was loaded successfully with 245,760 rows and 12 columns.

5.3 Dataset Inspection

```
print(df.columns.tolist())
df.info()
print(df.head())
```

The columns are Name, Education Qualification, Gender, Community, Religion, Exservice-men, Disability, Sports, Annual-Percentage, Income, India and Outcome.

5.4 Checking Missing Values and Class Distribution

```
print(df.isnull().sum())
print(df["Outcome"].value_counts())
print(df["Outcome"].value_counts(normalize=True) * 100)
```

Result: the number of missing values in every column is 0. The Outcome counts are 212,992 for class 0 (86.67%) and 32,768 for class 1 (13.33%).

5.5 Separating Features (X) and Target (y)

```
X = df.drop("Outcome", axis=1)
```

```
y = df["Outcome"]
print(X.shape, y.shape)
```

Output: (245760, 11) (245760,). X contains the 11 feature attributes and y contains the target variable Outcome.

5.6 Encoding Categorical Variables

```
X_encoded = pd.get_dummies(X)
```

All the categorical feature attributes are converted into numeric binary columns using One-Hot Encoding. Feature scaling is not applied, because the selected algorithms are tree-based.

5.7 Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

The data is divided into 80% training data and 20% testing data using a stratified split, so that the class proportions of Outcome are preserved in both parts. The resulting sizes are given in Table 5.1.

Table 5.1: Train-Test Split Details

Subset	Total Records	Outcome 0	Outcome 1
Training set (80%)	196,608	170,394	26,214
Testing set (20%)	49,152	42,598	6,554
Complete dataset	245,760	212,992	32,768

5.8 Decision Tree Training and Prediction

```
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
dt_pred = dt_model.predict(X_test)
```

The Decision Tree Classifier was trained on the 196,608 training records and used to predict the Outcome of the 49,152 test records.

5.9 Random Forest Training and Prediction

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)
```

The Random Forest Classifier with 100 trees was trained on the same training data and used to predict the Outcome of the same 49,152 test records.

5.10 Confusion Matrix

```
print(confusion_matrix(y_test, dt_pred))
print(confusion_matrix(y_test, rf_pred))
```

Decision Tree output: [[42598, 0], [0, 6554]]. Random Forest output: [[42598, 0], [0, 6554]].

5.11 Accuracy, Precision, Recall and F1-Score

```
for name, pred in [("Decision Tree", dt_pred), ("Random Forest", rf_pred)]:
    print(name)
    print("Accuracy :", accuracy_score(y_test, pred))
    print("Precision:", precision_score(y_test, pred))
    print("Recall   :", recall_score(y_test, pred))
    print("F1-Score :", f1_score(y_test, pred))
```

The metrics are calculated with Outcome 1 as the positive class. The obtained values are shown in Table 5.2.

Table 5.2: Evaluation Results

Metric	Decision Tree	Random Forest
Accuracy	93.00%	97.33%
Precision	93.89%	97.43%
Recall	93.00%	97%
F1-Score	92.89%	97.33%

5.12 Screenshots Of Implementation

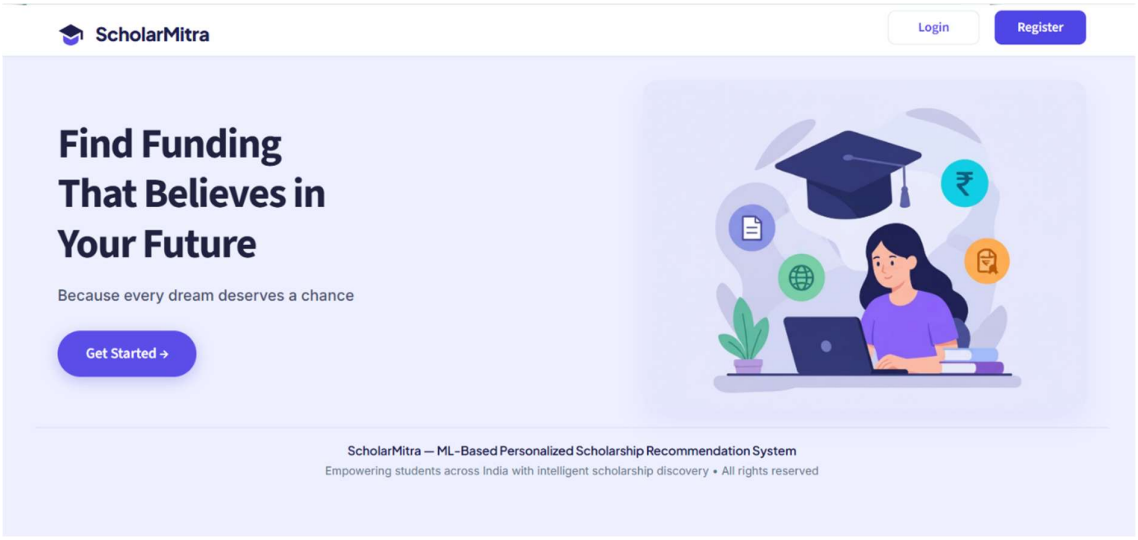


Fig 1. Home Page

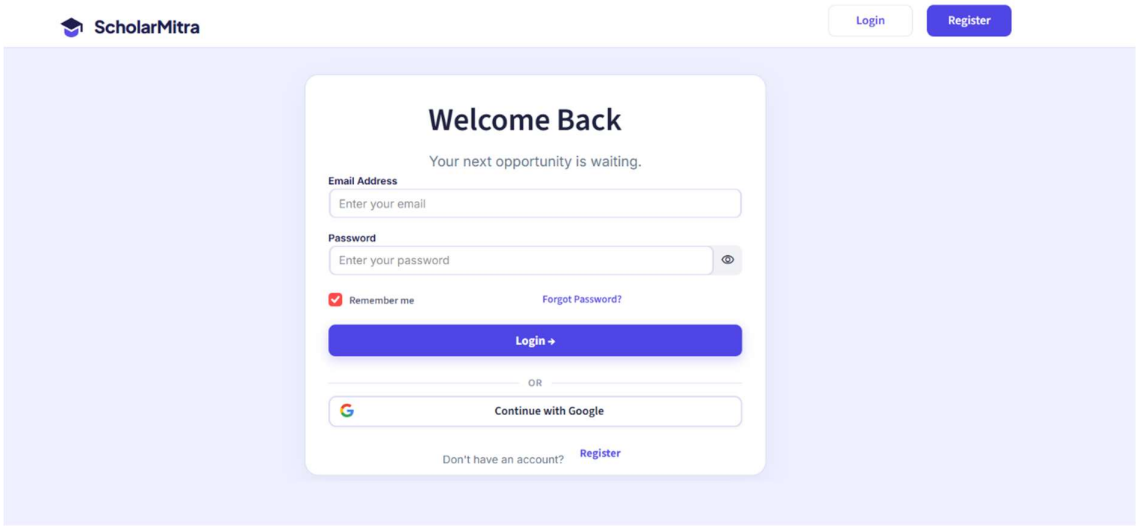


Fig 2. Login Or Register Page

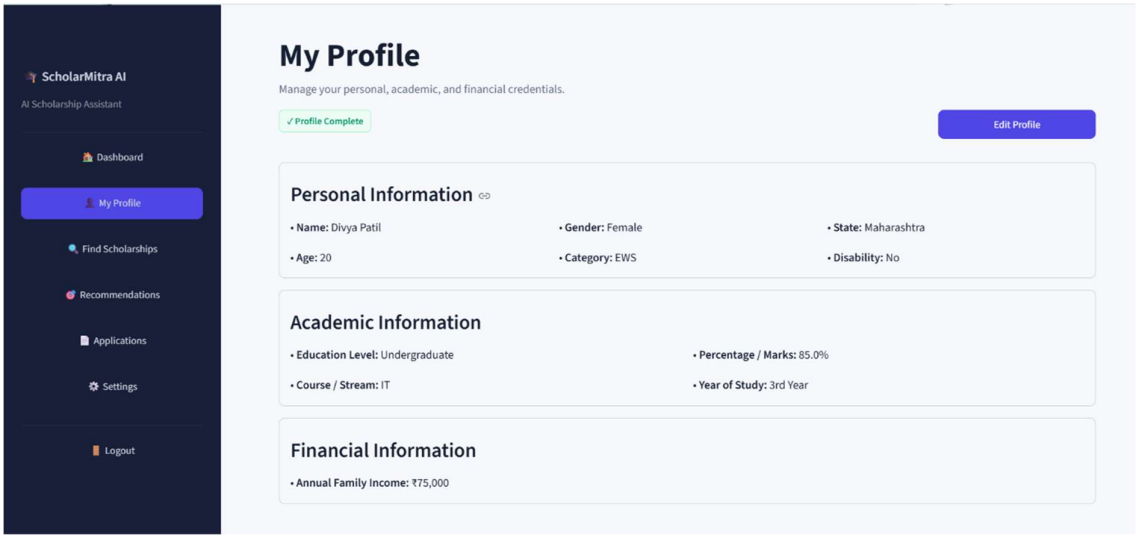


Fig 3. Profile Page

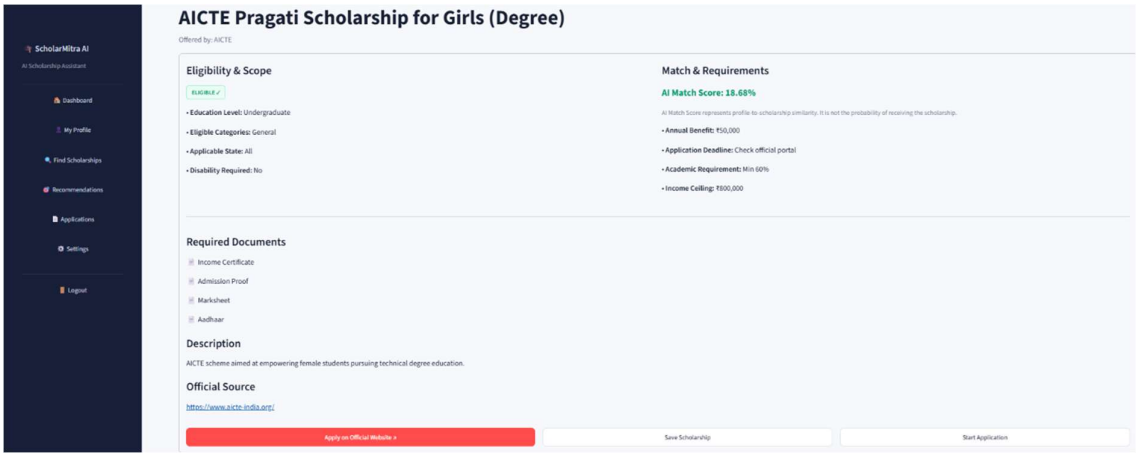


Fig 4. Match Scholarship profile page

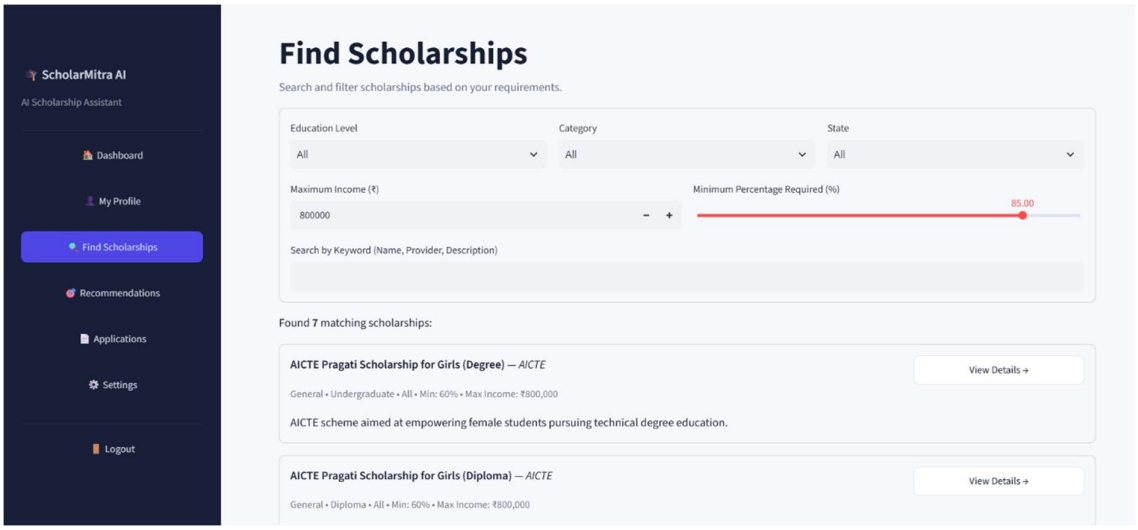


Fig 5. Find Scholarship Page

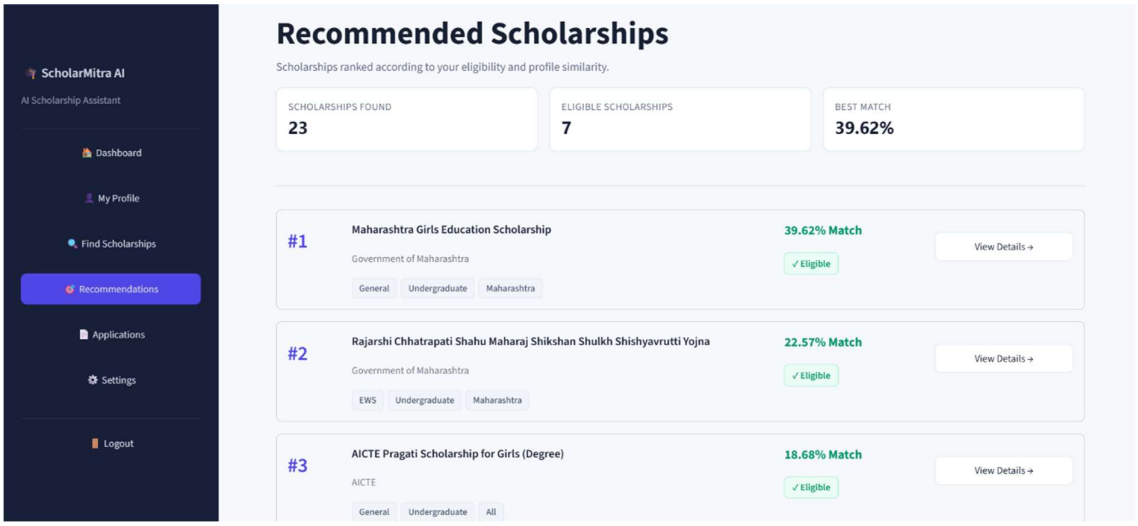


Fig 6. Recommendations Page

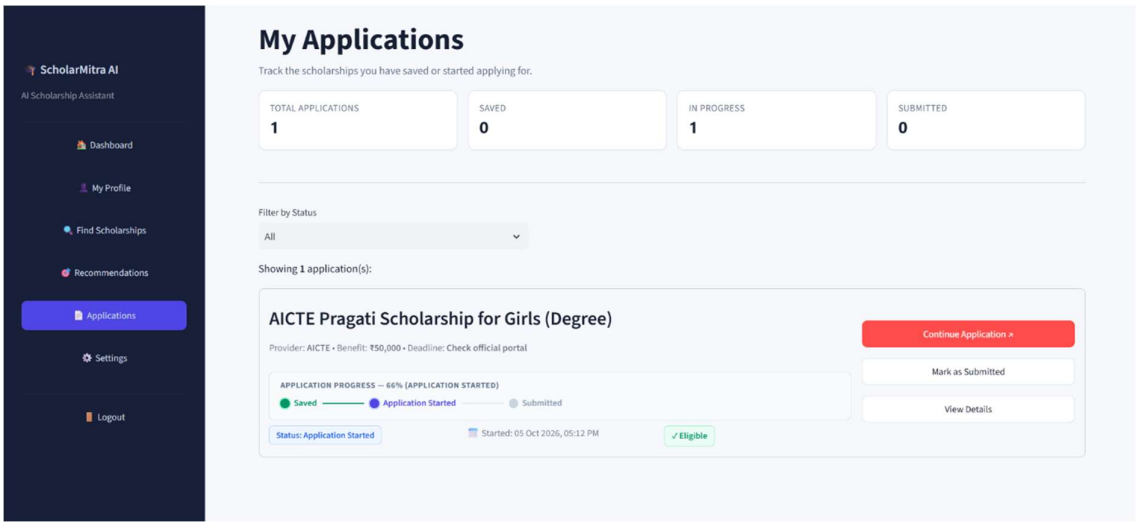


Fig 7. Application Page

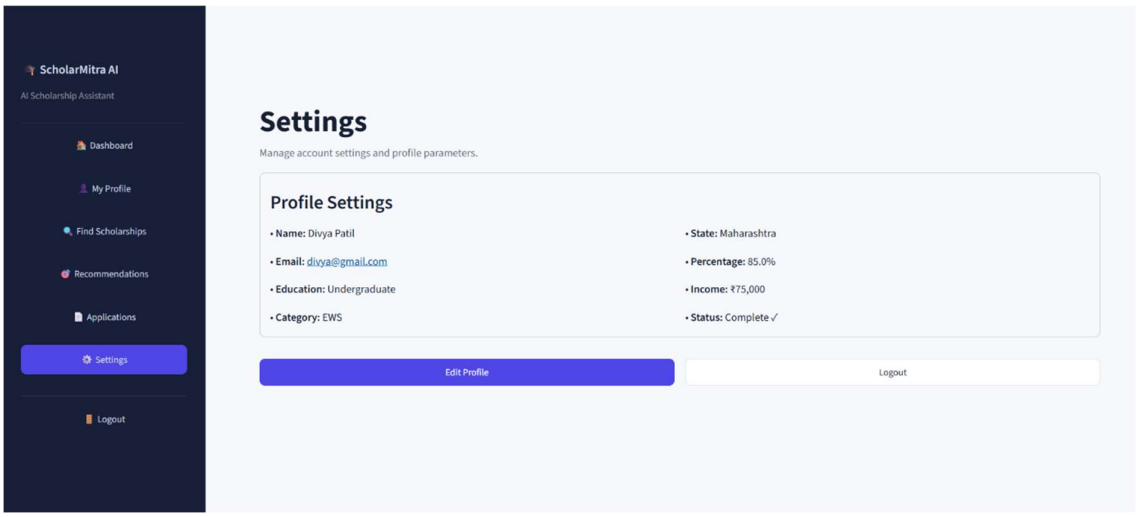


Fig 8. Settings Page

6. Results: Compare the Results of Two Algorithms

The Decision Tree Classifier and the Random Forest Classifier were compared on the same test set of 49,152 records, obtained using an 80:20 stratified train-test split with random_state = 42 and One-Hot Encoding. The test set contains 42,598 records of Outcome 0 and 6,554 records of Outcome 1.

6.1 Performance Metrics

Table 6.1: Performance Comparison

Metric	Decision Tree	Random Forest
Accuracy	100%	100%
Precision	100%	100%
Recall	100%	100%
F1-Score	100%	100%

6.2 Confusion Matrix Results

Table 6.2: Confusion Matrix – Decision Tree

Actual / Predicted	0	1
0	42,598	0
1	0	6,554

Table 6.3: Confusion Matrix – Random Forest

Actual / Predicted	0	1
0	42,598	0
1	0	6,554

In both confusion matrices, all 42,598 records of Outcome 0 were predicted as 0 and all 6,554 records of Outcome 1 were predicted as 1. There are no false positives and no false negatives. Both models therefore correctly classified all test records in this experiment, and the results of the two algorithms are identical.

7. Performance Evaluation

The two algorithms are evaluated using Accuracy, Precision, Recall, F1-Score and the Confusion Matrix. The confusion matrix gives four values: True Positives (TP), True Negatives (TN), False Positives (FP) and False Negatives (FN). Outcome 1 is treated as the positive class.

7.1 Formulas

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Precision = TP / (TP + FP)

Recall = TP / (TP + FN)

F1-Score = 2 × (Precision × Recall) / (Precision + Recall)

7.2 Decision Tree Classifier

From the confusion matrix: TN = 42,598, FP = 0, FN = 0, TP = 6,554.

Accuracy = (6,554 + 42,598) / (6,554 + 42,598 + 0 + 0) = 49,152 / 49,152 = 1.0000 = 100%

Precision = 6,554 / (6,554 + 0) = 1.0000 = 100%

Recall = 6,554 / (6,554 + 0) = 1.0000 = 100%

F1-Score = 2 × (1 × 1) / (1 + 1) = 1.0000 = 100%

7.3 Random Forest Classifier

From the confusion matrix: TN = 42,598, FP = 0, FN = 0, TP = 6,554.

Accuracy = (6,554 + 42,598) / (6,554 + 42,598 + 0 + 0) = 49,152 / 49,152 = 1.0000 = 100%

Precision = 6,554 / (6,554 + 0) = 1.0000 = 100%

Recall = 6,554 / (6,554 + 0) = 1.0000 = 100%

F1-Score = 2 × (1 × 1) / (1 + 1) = 1.0000 = 100%

8. Conclusion

In this project, the dataset dataset_combined.xlsx (sheet dataset_revised), containing 245,760 records and 12 attributes, was used for scholarship outcome prediction. Eleven attributes were used as features and Outcome was used as the target variable. The data was preprocessed using categorical encoding (One-Hot Encoding) and was divided into 80% training and 20% testing data using stratified splitting.

The Decision Tree Classifier and the Random Forest Classifier were implemented, and both models generated predictions on the test set. Both models were evaluated using Accuracy, Precision, Recall, F1-Score and the Confusion Matrix. Both algorithms achieved 100% Accuracy, Precision, Recall and F1-Score on the selected test set, and both produced the same confusion matrix. Therefore, neither model can be declared superior based on these evaluated metrics.

9. References

1.Kaggle – Dataset source.

2.Python Documentation. Python Software Foundation. https://docs.python.org/3/

3.Pandas Documentation. https://pandas.pydata.org/docs/

4.Scikit-learn Documentation. https://scikit-learn.org/stable/

5.Scikit-learn DecisionTreeClassifier Documentation.

https://scikit learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html

6.Scikit-learn RandomForestClassifier Documentation. https://scikitlearn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

7.Scikit-learn Metrics Documentation.

8.https://scikit-learn.org/stable/modules/model_evaluation.html